



Digital Design

Chapter 2: Combinational Logic Design

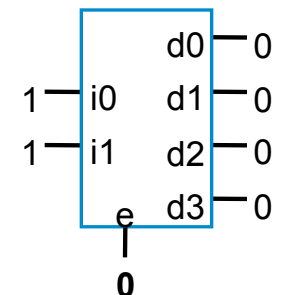
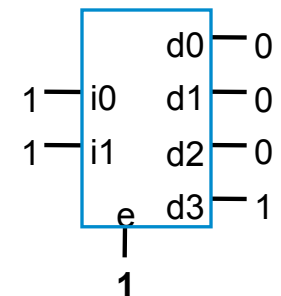
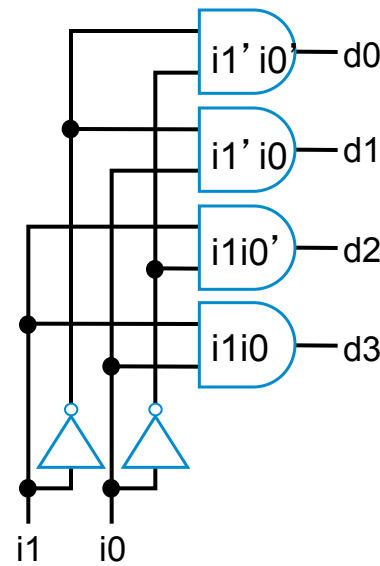
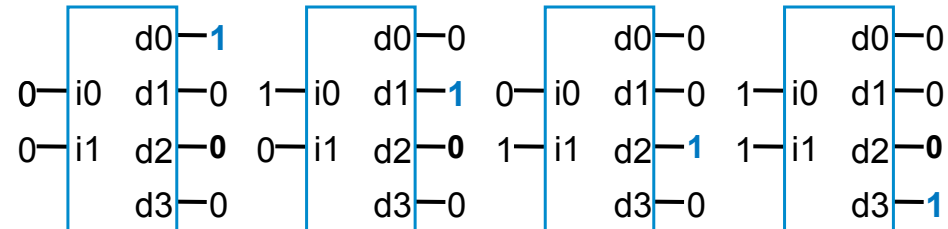
Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

Decoders

- **Decoder:** Popular combinational logic building block, in addition to logic gates
 - Converter from input binary number to **one high output**
- 2-input decoder: four possible input binary numbers
 - So has four outputs, one for each possible input binary number
- Internal design
 - AND gate for each output to detect input combination
- **Decoder with enable signal e**
 - Outputs all 0 if $e=0$
 - Regular behavior if $e=1$
- n -input decoder: 2^n outputs



Behavioral Verilog Decoder

```
module Dcd2x4(I1, I0, D3, D2, D1, D0);

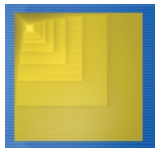
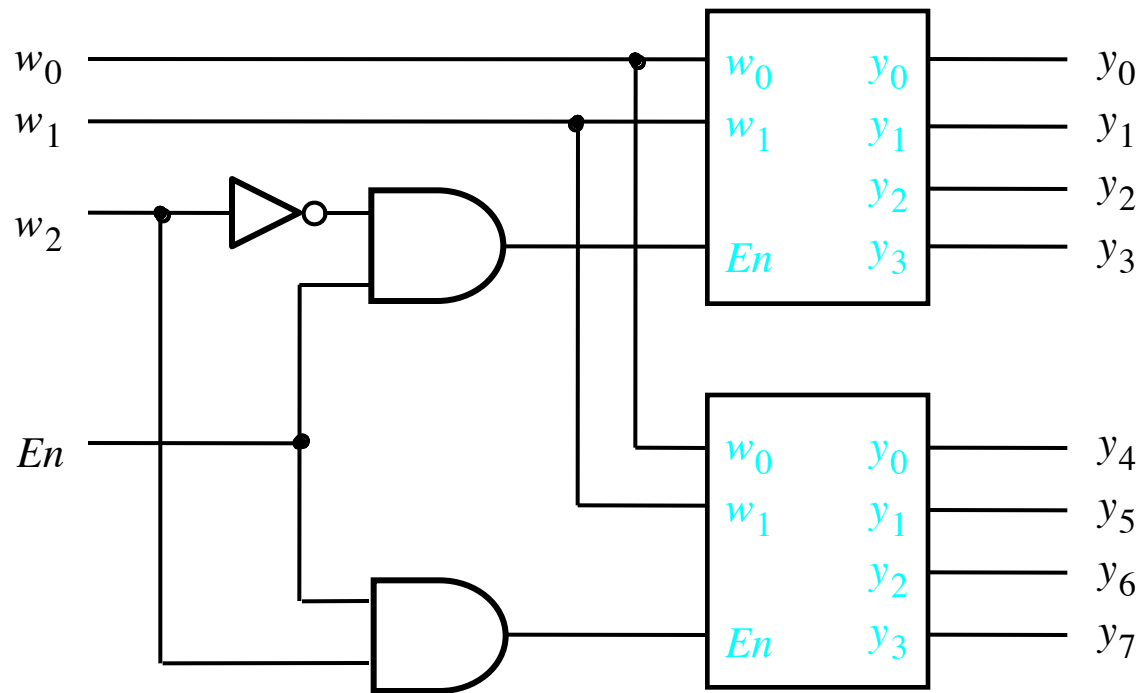
    input I1, I0;
    output D3, D2, D1, D0;

    reg D3, D2, D1, D0;

    always @(I1, I0)
    begin
        if (I1==0 && I0==0)
            begin
                D3 <= 0; D2 <= 0; D1 <= 0; D0 <= 1;
            end
        else if (I1==0 && I0==1)
            begin
                D3 <= 0; D2 <= 0; D1 <= 1; D0 <= 0;
            end
        else if (I1==1 && I0==0)
            begin
                D3 <= 0; D2 <= 1; D1 <= 0; D0 <= 0;
            end
        else
            begin
                D3 <= 1; D2 <= 0; D1 <= 0; D0 <= 0;
            end
        end
    end
endmodule
```

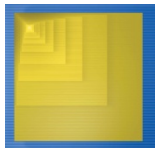
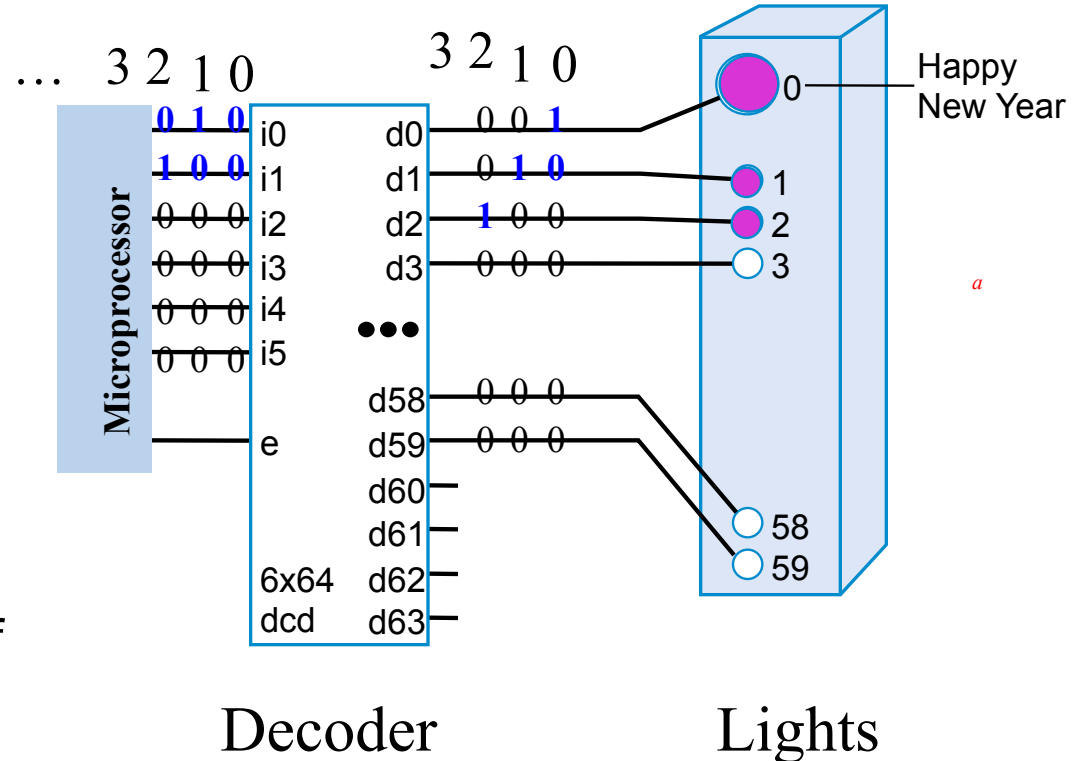


A 3-to-8 decoder using two 2-to-4 decoder with enable



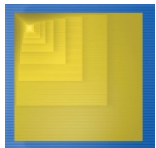
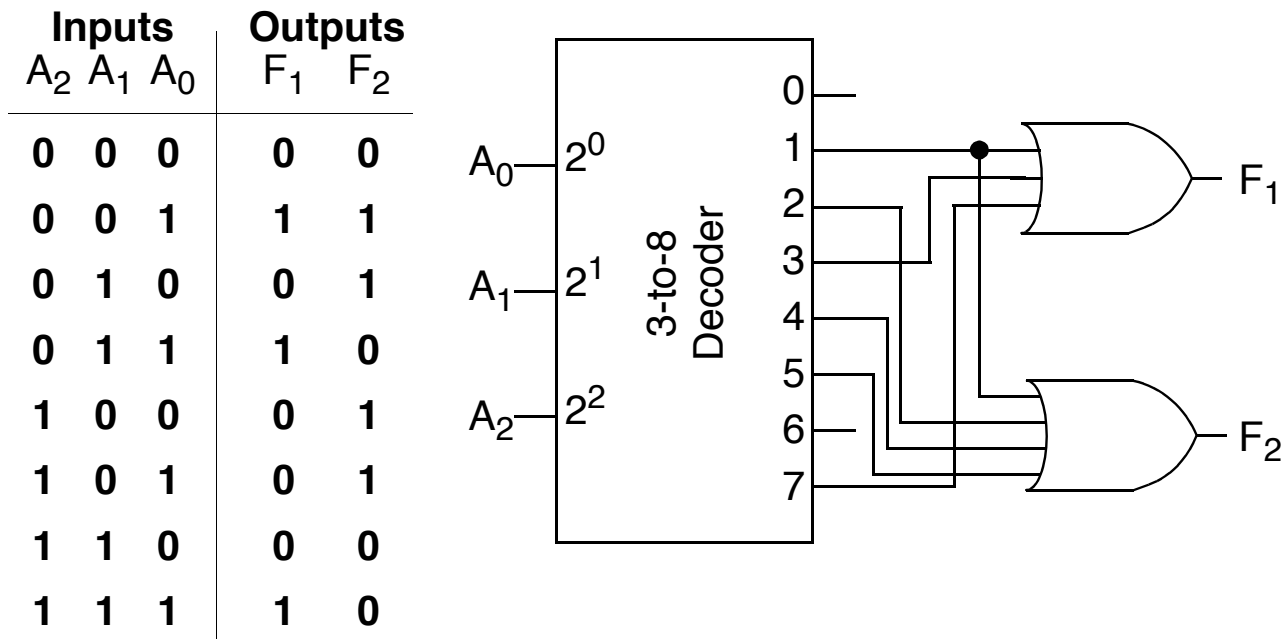
Decoder Example

- New Year's Eve Countdown Display
 - Microprocessor counts from 59 down to 0 in binary on 6-bit output
 - (6 bits can generate $2^6 = 64$ numbers)
 - Want to illuminate one of 60 lights for each binary number
 - Use 6x64 decoder
 - 4 outputs unused



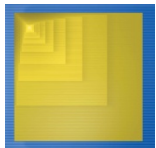
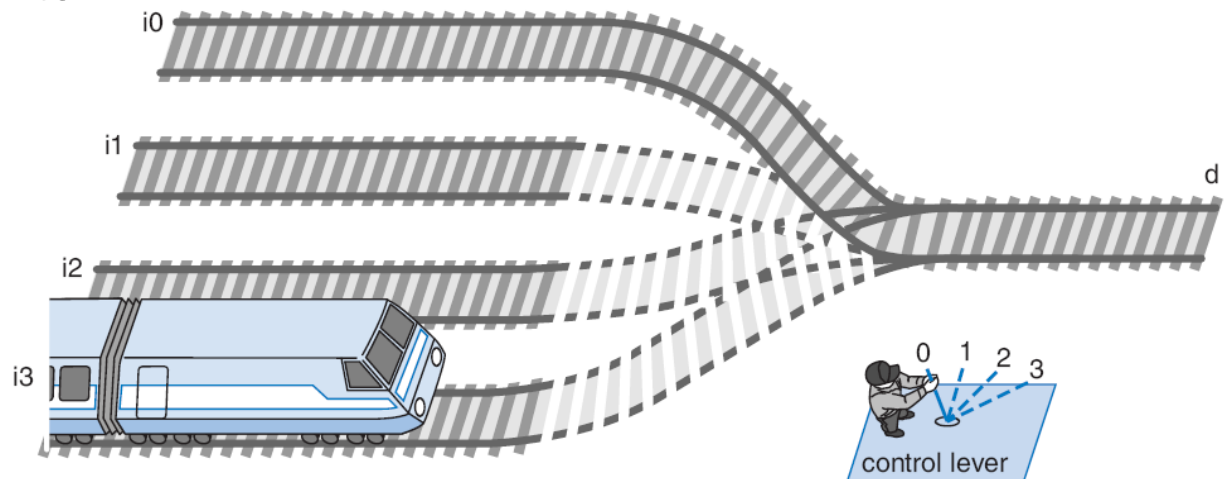
Decoder implements any Boolean Function

- Any Boolean function can be implemented using a **decoder** and **OR** gates by ORing together the function's **minterms**.

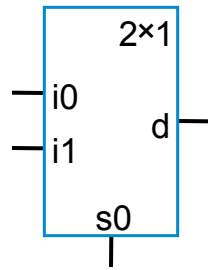


Multiplexor (Mux)

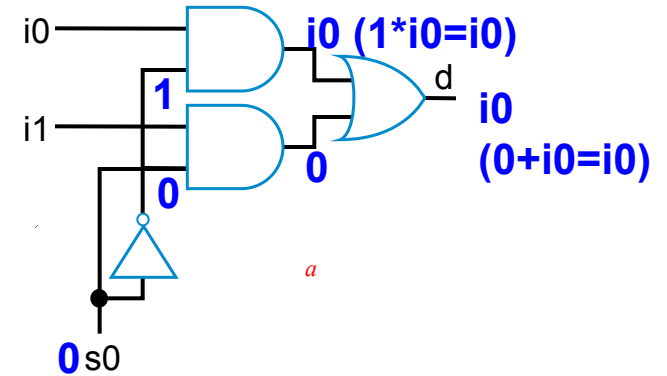
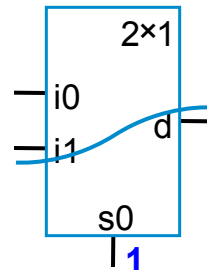
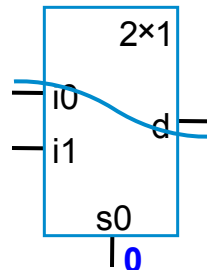
- Mux: Another popular combinational building block
 - Routes one of its N data inputs to its **one output**, based on binary value of select inputs
 - 4 input mux $\rightarrow$ needs 2 select inputs to indicate which input to route through
 - 8 input mux $\rightarrow$ 3 select inputs
 - N inputs $\rightarrow \log_2(N)$ selects
 - Like a railyard switch



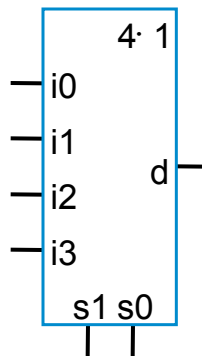
Mux Internal Design



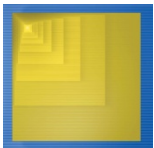
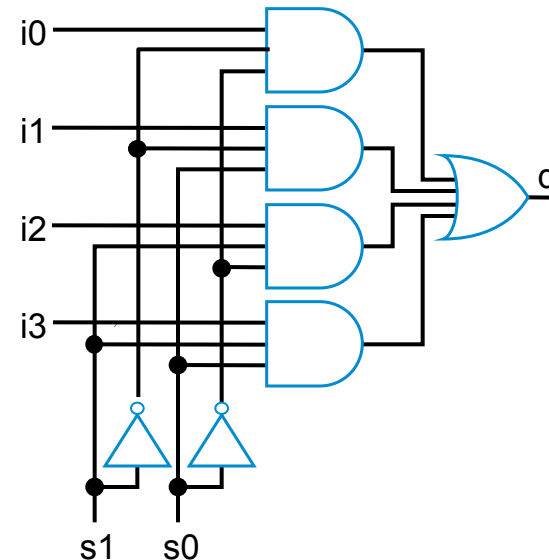
2x1 mux



Truth table for a mux

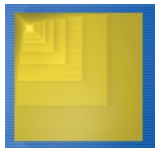
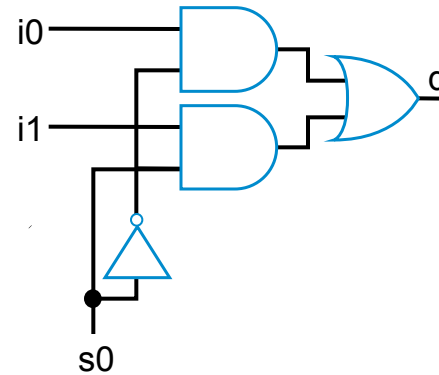


4x1 mux



Structural (Gate Level) Verilog Mux

```
module mux2(d,i0,i1,s0);  
output d;  
input i0,i1,s0;  
//internal wires  
wire sbar,y1,y2;  
    not g1(sbar,s0);  
    and g2(y1,i0,sbar);  
    and g3(y2,i1,s0);  
    or g4(d,y1,y2);  
endmodule
```



Behavioral Verilog Mux

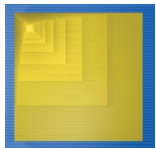
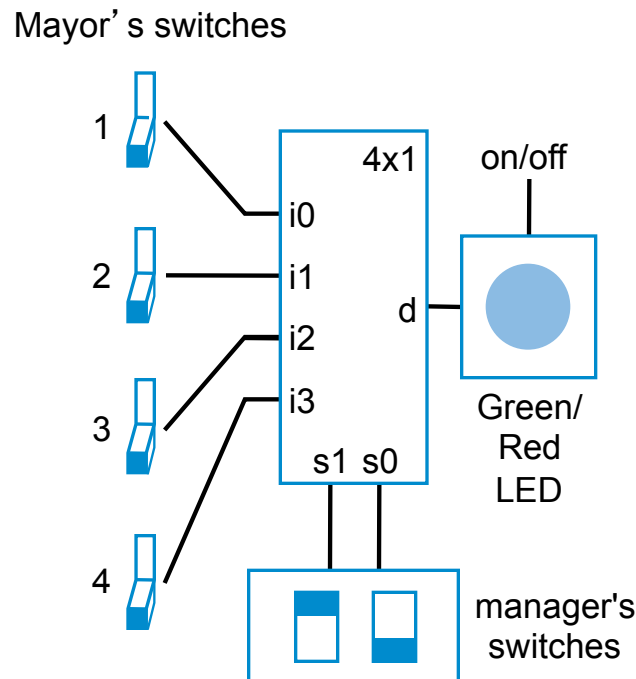
```
module mux4x1(S0, S1, I0, I1, I2, I3, O);
    input I0, I1, I2, I3;
    input S0, S1;
    output O;
    reg O;

    always @(I0, I1, I2, I3, S0, S1)
    begin
        if( S1==0 && S0==0 )
            O <= I0;
        else if( S1==0 && S0==1 )
            O <= I1;
        else if( S1==1 && S0==0 )
            O <= I2;
        else
            O <= I3;
    end
endmodule
```



Mux Example

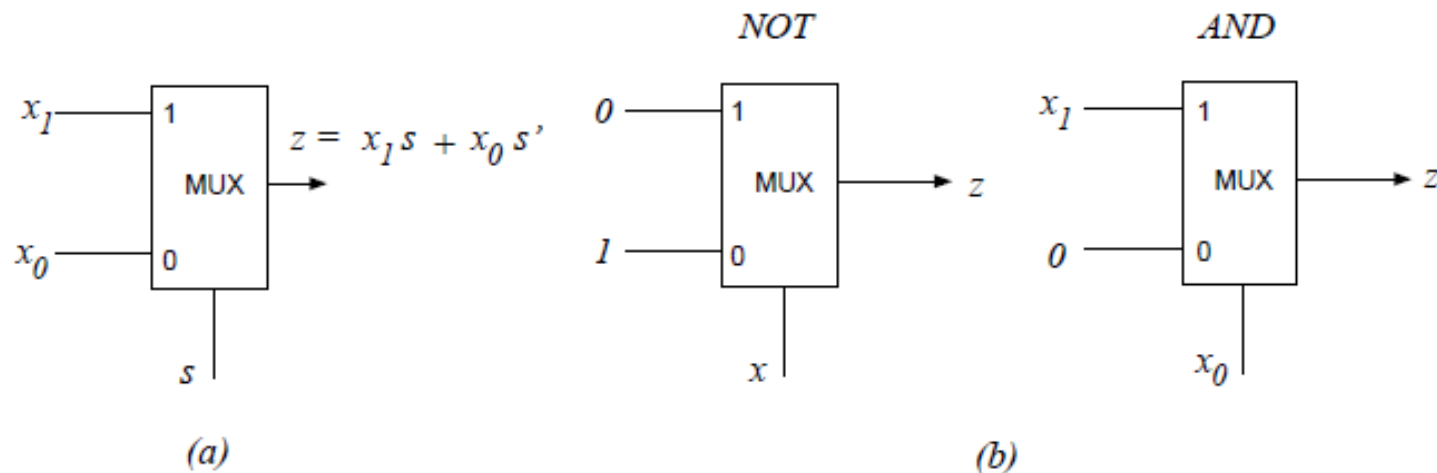
- City mayor can set four switches up or down, representing his/her vote on each of four proposals, numbered 0, 1, 2, 3
- City manager can display any such vote on large green/red LED (light) by setting two switches to represent binary 0, 1, 2, or 3
- Use 4x1 mux



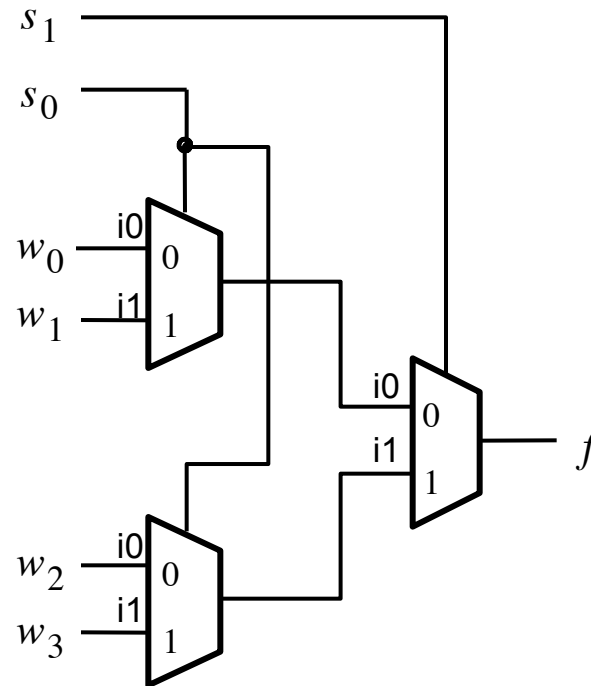
Mux

- 2-INPUT *multiplexer* (MUX): $z = \text{MUX}[x_1, x_0, s] = x_1 \cdot s + x_0 s'$
- SET {MUX } IS UNIVERSAL (constants 0 and 1 available)

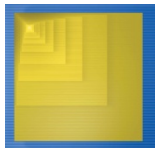
$$\text{NOT}(x) = \text{MUX}[0, 1, x] = 0 \cdot x + 1 \cdot x' = x'$$
$$\text{AND}(x_1, x_0) = \text{MUX}[x_1, 0, x_0] = x_1 x_0 + 0 \cdot x_0' = x_1 x_0$$



Using 2-to-1 multiplexers to build a 4-to-1 multiplexer

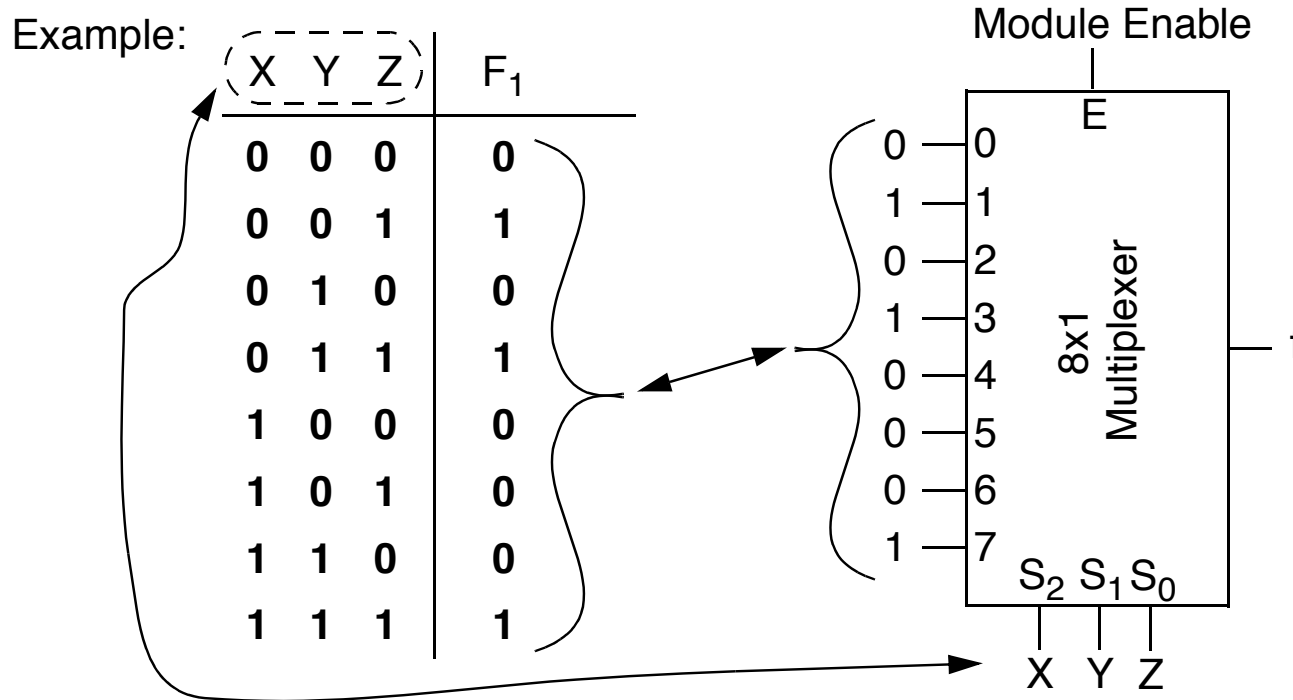


Do not forget to put labels on select, inputs and output.
That is, $s_0, s_1, i_0, i_1 \dots$ etc.



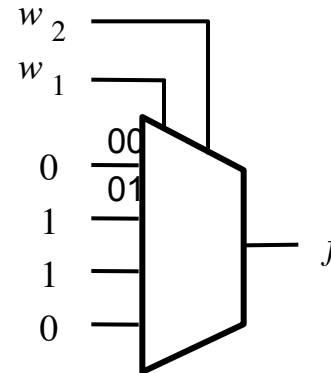
Mux implements any Boolean Function

- Any Boolean function can be implemented using MUX by setting the inputs corresponding to the function and the selectors as the variables.



Synthesis of a logic function using multiplexers

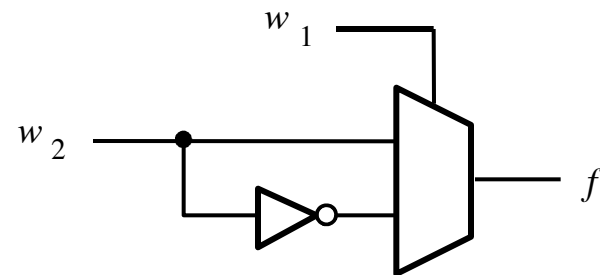
w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0



(a) Implementation using a 4-to-1 multiplexer

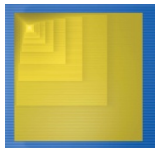
w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0

w_1	f
0	w_2
1	$\overline{w_2}$



(b) Modified truth table

(c) Circuit



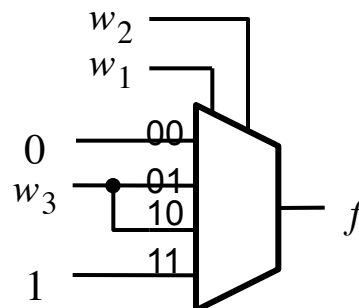
Three-input majority function using 4-to-1 mux

w_1	w_2	w_3	f		w_1	w_2	f
0	0	0	0	}	0	0	0
0	0	1	0		0	1	w_3
0	1	0	0		1	0	w_3
0	1	1	1		1	1	1
1	0	0	0	}			
1	0	1	1				
1	1	0	1	}			
1	1	1	1				

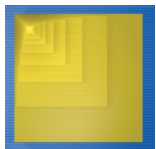
(a) Modified truth table

Important:

- Use a truth table
- Label the mux inputs properly.
- For example;
 $w_1 w_2$ where w_1 is most significant bit
 Inputs 00, 01, 10, 11



(b) Circuit

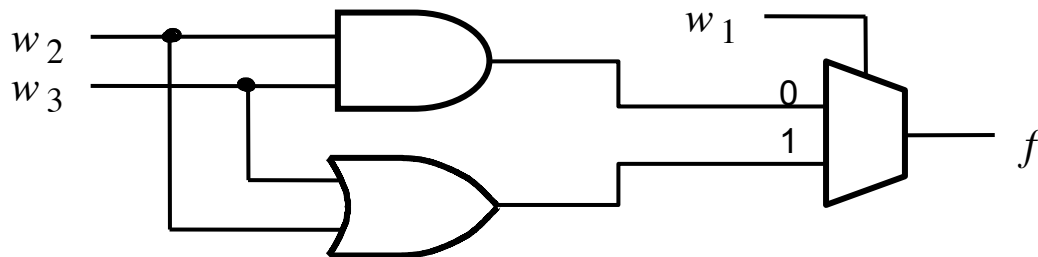


Three-input majority function using a 2-to-1 MUX

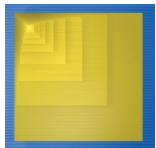
w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

w_1	f
0	$w_2 w_3$
1	$w_2 + w_3$

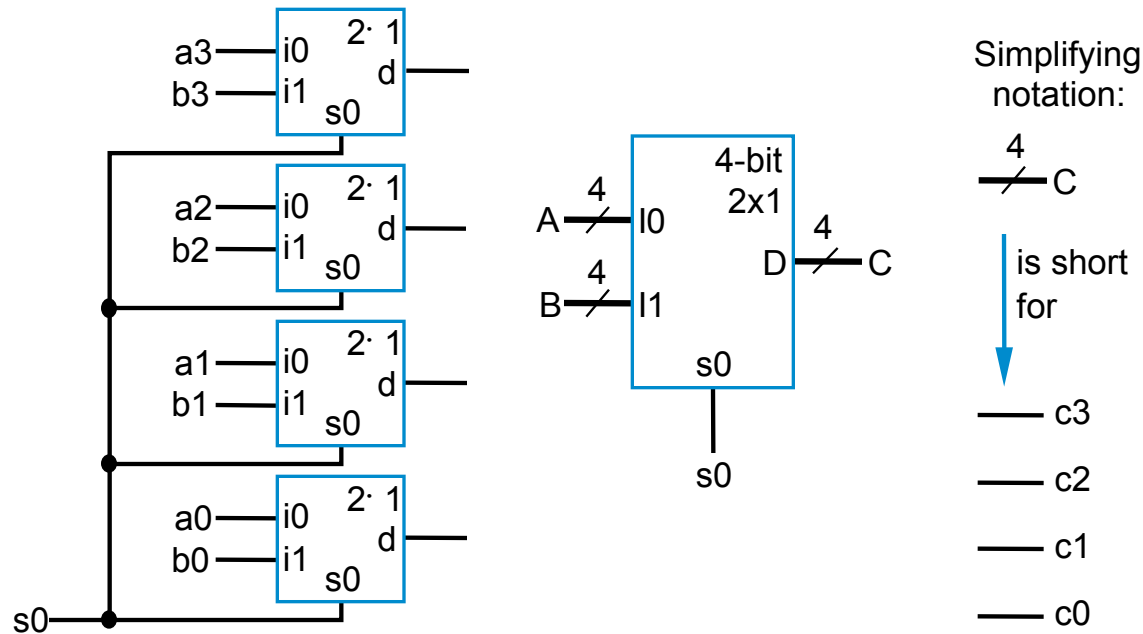
(b) Truth table



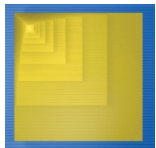
(b) Circuit



Muxes Commonly Together -- N-bit Mux



- Ex: Two 4-bit inputs, A ($a_3 a_2 a_1 a_0$), and B ($b_3 b_2 b_1 b_0$)
 - 4-bit 2x1 mux (just four 2x1 muxes sharing a select line) can select between A or B

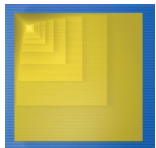


Shannon's Expansion

- Any Boolean function $f(w_1, \dots, w_n)$ can be written in the form

$$f(w_1, w_2, \dots, w_n) = \underbrace{\bar{w}_1 \cdot f(0, w_2, \dots, w_n)}_{\text{cofactor}} + w_1 \cdot \underbrace{f(1, w_2, \dots, w_n)}_{\text{cofactor}}$$

- Shannon's expansion can be used multiple times to obtain canonical sum of minterms (or product of maxterms)
- Multiplexers can be used to implement any Boolean function based on Shannon's expansion.



Shannon's Expansion

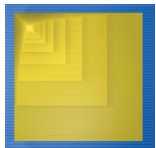
$$\begin{aligned}f(w_1, w_2, \dots, w_n) &= w'_1 \cdot f(0, w_2, \dots, w_n) + w_1 \cdot f(1, w_2, \dots, w_n) \\&= w'_1 f_{w'_1} + w_1 f_{w_1}\end{aligned}$$

Suitable for 2-to-1 mux. Select input for Mux is w_1

$$\begin{aligned}f(w_1, w_2, \dots, w_n) &= w'_1 w'_2 \cdot f(0, 0, w_3, \dots, w_n) + \\&\quad w'_1 w_2 \cdot f(0, 1, w_3, \dots, w_n) + \\&\quad w_1 w'_2 \cdot f(1, 0, w_3, \dots, w_n) + \\&\quad w_1 w_2 \cdot f(1, 1, w_3, \dots, w_n)\end{aligned}$$

Suitable for 4-to-1 mux. Select inputs for Mux are $w_1 w_2$

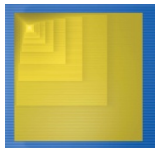
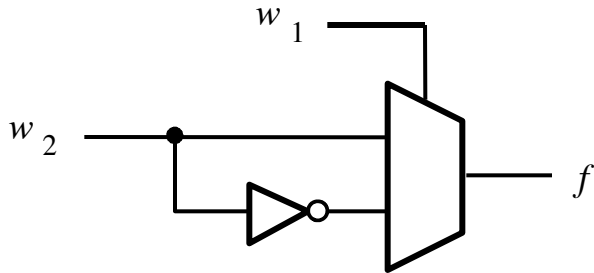
What happens if expansion is done for all n variables ? *Canonical SOP form*



Shannon's Expansion

$$\begin{aligned} f(w_1, w_2) &= \bar{w}_1 \cdot w_2 + w_1 \bar{w}_2 \\ &= \bar{w}_1 \cdot f(0, w_2) + w_1 \cdot f(1, w_2) \\ &= \bar{w}_1 \cdot w_2 + w_1 \bar{w}_2 \end{aligned}$$

Same function after Shannon's expansion



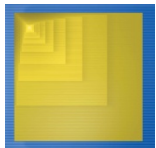
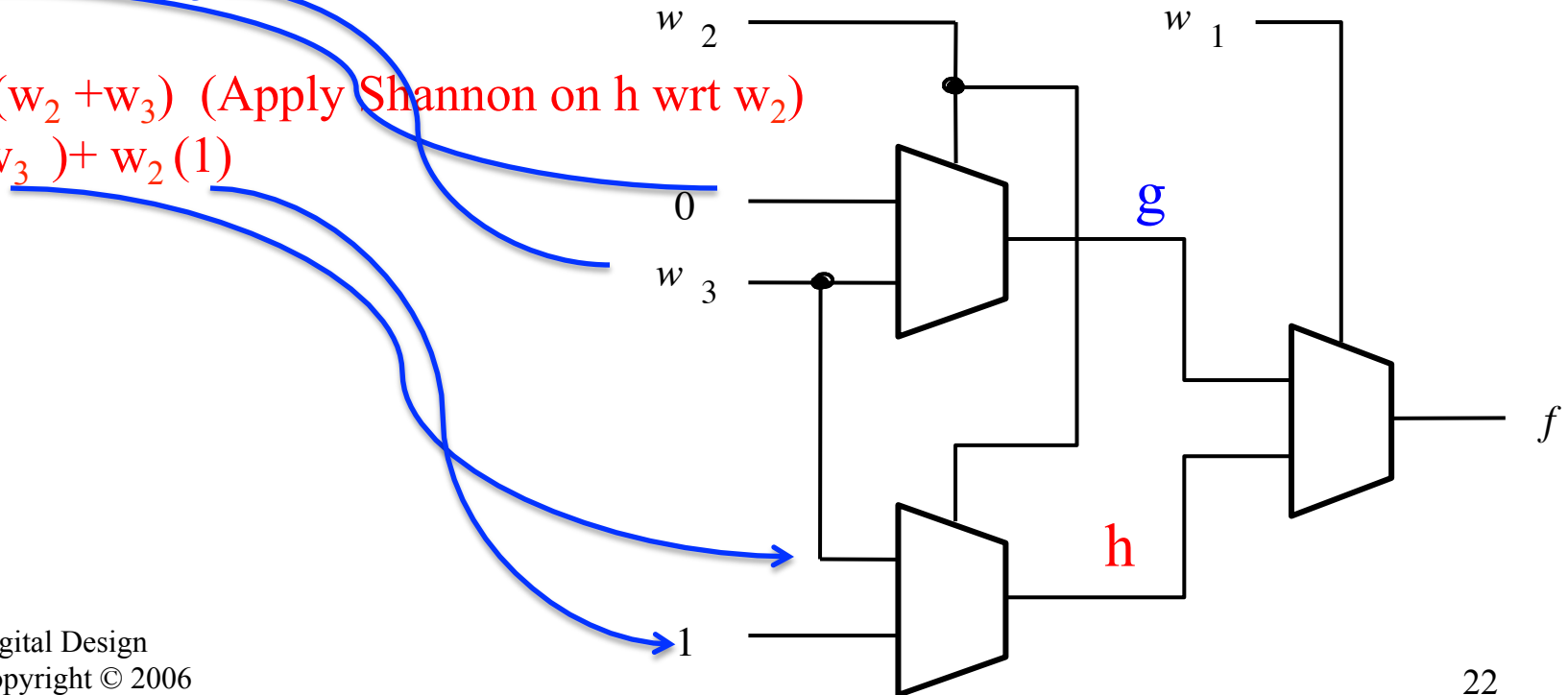
Three-input majority function using Shannon's Expansion

$$f = w_1' w_2 w_3 + w_1 w_2 + w_1 w_3 \quad (\text{Apply Shannon on } f \text{ with respect to } w_1) \\ = w_1' (w_2 w_3) + w_1 (w_2 + w_3)$$

Easier to apply Shannon separately on cofactors wrt w_2

$$\text{Let } g = (w_2 w_3) \quad (\text{Apply Shannon on } g \text{ wrt } w_2) \\ = w_2' (0) + w_2 (w_3)$$

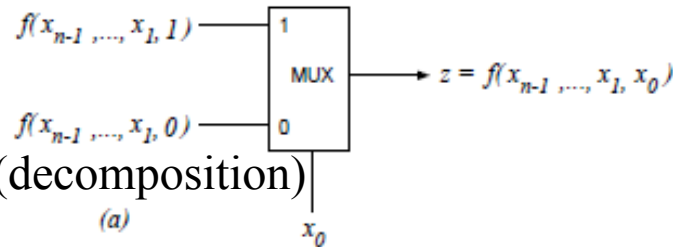
$$\text{Let } h = (w_2 + w_3) \quad (\text{Apply Shannon on } h \text{ wrt } w_2) \\ = w_2' (w_3) + w_2 (1)$$



MUXES

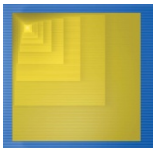
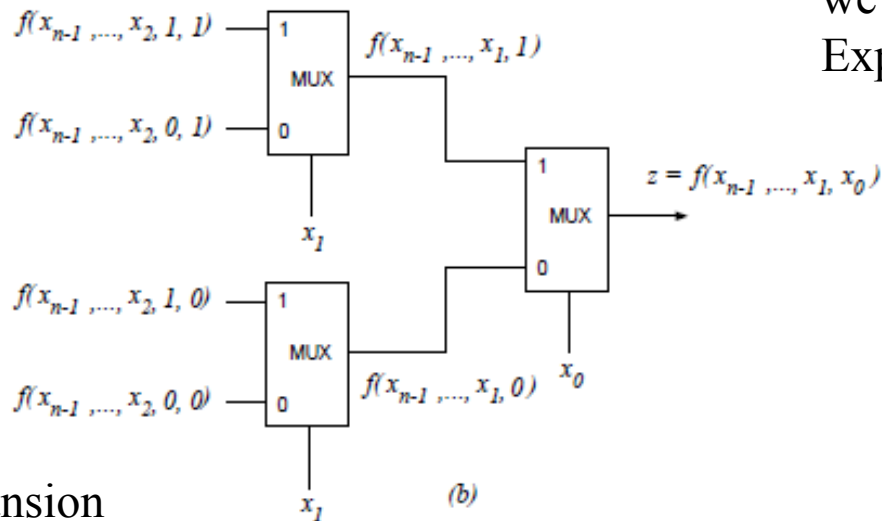
OBTAIN A TREE OF MULTIPLEXERS BY REPEATED USE OF SD

One level of expansion (decomposition)



For a function with N variables we need $2^N - 1$ muxes for N level Expansion (binary tree)

Two levels of expansion



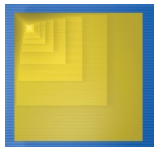
Mux

- IMPLEMENT $f(x_3, x_2, x_1, x_0) = z = x_3(x_1 \oplus x_2x_0)$ WITH MUX TREE
- DECOMPOSE WITH RESPECT TO x_2, x_1, x_0

Use the tree of muxes in previous slide

$$\begin{aligned}f(x_3, 0, 0, 0) &= 0 & f(x_3, 0, 0, 1) &= 0 \\f(x_3, 0, 1, 0) &= x_3 & f(x_3, 0, 1, 1) &= x_3 \\f(x_3, 1, 0, 0) &= 0 & f(x_3, 1, 0, 1) &= x_3 \\f(x_3, 1, 1, 0) &= x_3 & f(x_3, 1, 1, 1) &= x_3\end{aligned}$$

- ELIMINATE REDUNDANT MUXes



Ordering of Variables in subtrees affects the number of MUXes

Analyze these two MUX trees:

$$z = x_0 (f) + x_0' (g)$$

$$f = x_1 x_3 + x_1' (h)$$

$$h = x_2 x_3$$

$$f = x_1 x_3 + x_1' x_2 x_3$$

$$g = x_1 x_3$$

$$z = x_0 (x_1 x_3 + x_1' x_2 x_3) + x_0' x_1 x_3$$

$$z = x_3 (x_0 x_1 + x_0 x_1' x_2 + x_0' x_1)$$

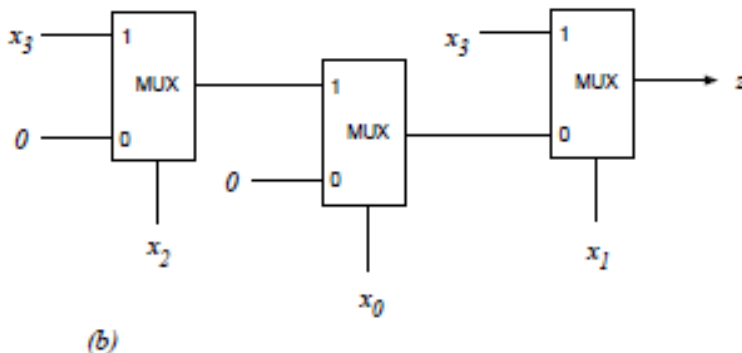
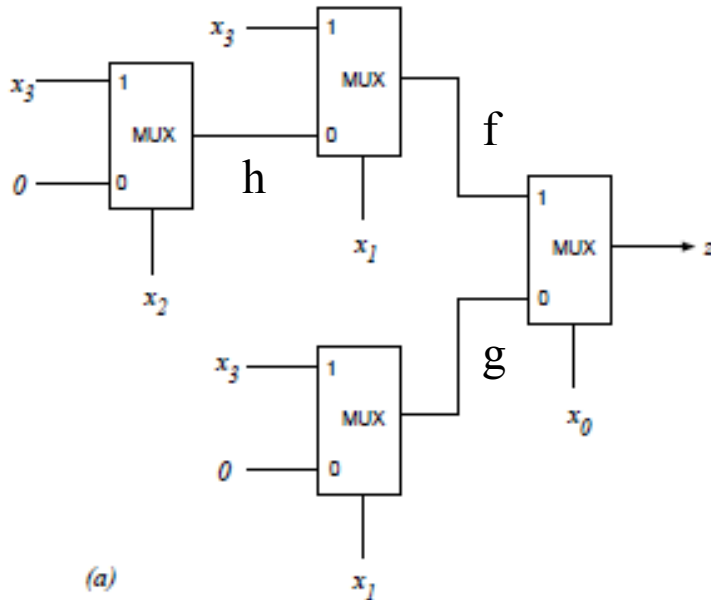
$$z = x_3 (x_1 + x_0 x_1' x_2)$$

$$z = x_3 (x_1 + x_0)(x_1 + x_1')(x_1 + x_2)$$

$$z = x_3 (x_1 + x_0)(x_1 + x_2)$$

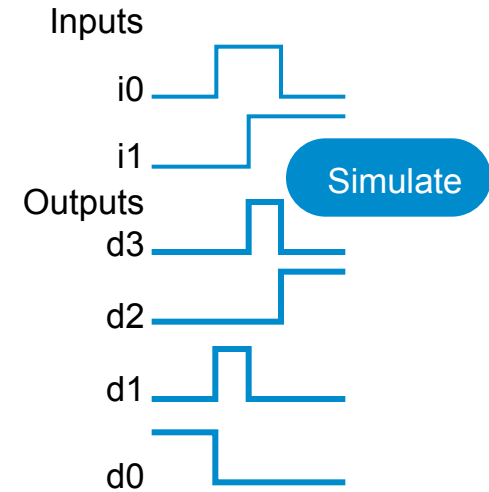
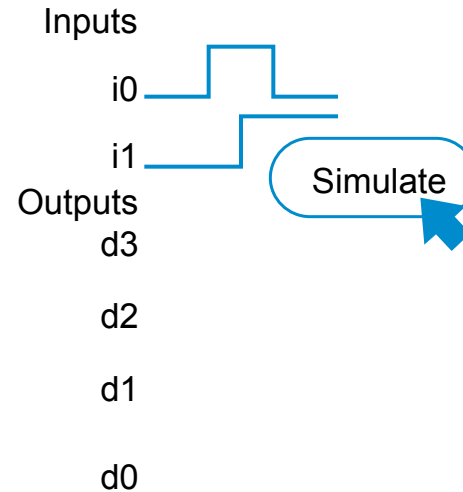
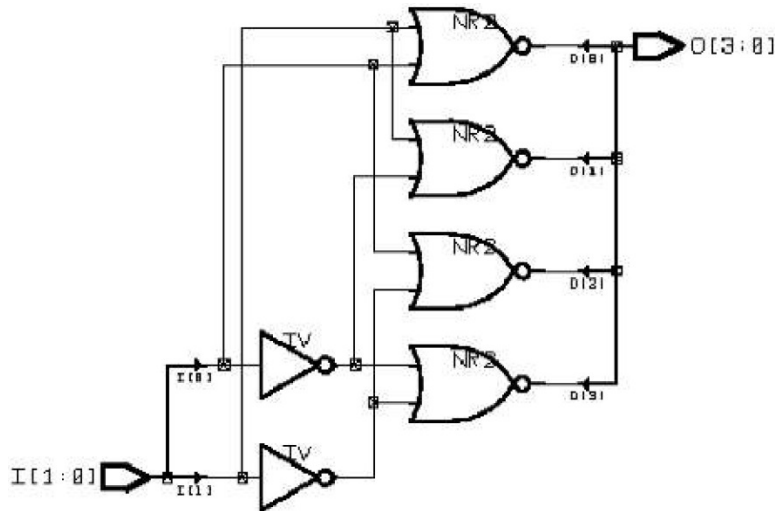
$$z = x_3 (x_1 + x_0 x_2)$$

Same function two different MUX trees

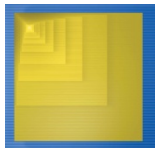


Additional Considerations

Schematic Capture and Simulation

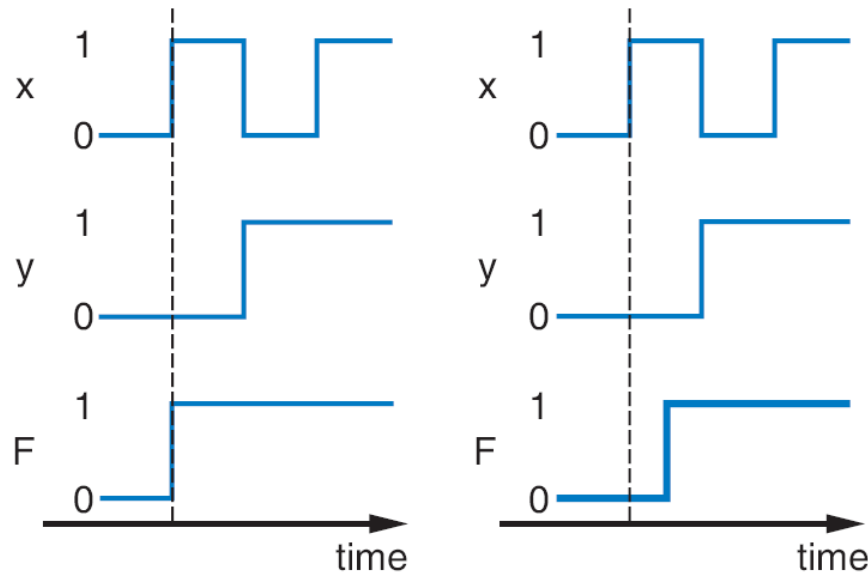
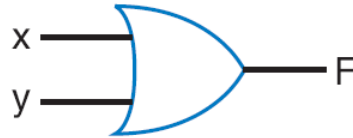


- **Schematic capture**
 - Computer tool for user to capture logic circuit graphically
- **Simulator**
 - Computer tool to show what circuit outputs would be for given inputs
 - Outputs commonly displayed as **waveform**

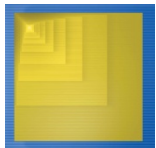


Additional Considerations

Non-Ideal Gate Behavior -- Delay



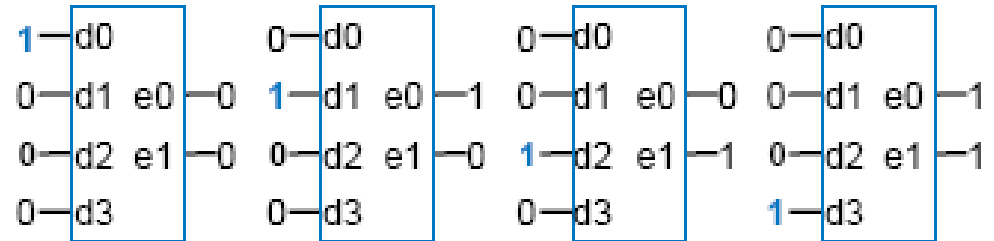
- Real gates have some delay
 - Outputs don't change immediately after inputs change



Additional Considerations

Encoders

- **Encoder:** Combinational logic building block with opposite functionality of decoder
 - Outputs binary encoding for in signal that is 1
 - 4x2 encoder would have four inputs and 2 outputs



What if two inputs are 1?

- Can use a **priority encoder**
- Gives priority to the highest input that is 1, and outputs binary encoding for that input
- Example: If d3=1 and d1=1, will output e0=1 and e1=1 because d3 has priority

